Document number: P1141R2

Ville Voutilainen
Thomas Köppe
Andrew Sutton
Herb Sutter
Gabriel Dos Reis
Bjarne Stroustrup
Jason Merrill
Hubert Tong
Eric Niebler
Casey Carter
Tom Honermann
Erich Keane
Walter E. Brown
Michael Spertus
Richard Smith
2018-11-09

Yet another approach for constrained declarations

Abstract

We propose a short syntax for the constrained declaration of function parameters, function return types and variables. The new syntax is a "constrained `auto`", e.g. `void sort(Sortable auto& c);`.

Contents

Revision history

- P1141R2 (this document): Added formal wording for parts 1, 3, and 4. Parts 2 and 5 are not being proposed.
- P1141R1: Added discussion in Parts 2 and 5 on *return-type-requirement*s and in Part 4 on variadic concepts.
- P1141R0: Initial proposal.

Proposal summary

This paper proposes three things:

1. A syntax for constrained declarations that is practically a "constrained `auto`"; the principle being "wherever `auto` goes, a `Constraint auto` can also (non-recursively) go". The semantics are to deduce like `auto` and additionally check a constraint. In a nutshell,

```
void f(Sortable auto x);
Sortable auto f();      // #1
```

```
Sortable auto x = f();  // #2
template <Sortable auto N> void f();
```

and all combined:

```
template <Sortable auto N> Sortable auto f(Sortable auto x)
{
    Sortable auto y = init;
}
```

An unconstrained version of that is:

```
template <auto N> auto f(auto x)
{
    auto y = init;
}
```

So, this proposal includes `auto`-typed parameters for functions, which we already allow for lambdas.

2. Simplifying (and thus restricting) the rules in [temp.param]/10, so that `template <Sortable S>` always means that `S` is a type parameter, and `template <Sortable auto S>` always means that `S` is a non-type parameter. Template template-parameters are no longer supported in this short form. Moreover, `Sortable` is restricted to be a concept that takes a type parameter or type parameter pack; non-type and template concepts are no longer supported in this short form.

3. Changing the meaning of parameter packs, so that `template <Sortable ...T>` means `requires Sortable<T> && ... && true`, and not `requires Sortable<T...>`.

`Sortable` is a "type concept" in all the examples of this summary.

This paper specifically does *not* propose

- any new lead-in syntax for templates, or
- a new syntax for introducing names for placeholder types, or
- a shortcut syntax for applying multiple constraints to a placeholder type.

The idea of this approach is to provide a syntax that

- works for constrained function parameters, constrained return types, constrained variables, and type-constrained non-type template parameters;
- avoids inventing many adventurous new things;
- in particular, avoids inventing new type sigils;
- does not clash with explicit template instantiations; and
- is compatible with what we already have in polymorphic lambdas, and makes functions uniform with them.

The previous revision of this paper (P1141R1) also proposed (in Part 2) an optional relaxation where the `auto` would be optional for the cases #1 and #2 illustrated above, and (in Part 5) a change of the meaning of `-> Concept auto`. However, EWG decided to propose only parts 1, 3, and 4.

Proposal details

Part 1: "Constrained `auto`"

The approach proposed here borrows a subset of P0807R0 An Adjective Syntax for Concepts. The idea is that we don't try to come up with a notation that does everything that P0807 does; in particular, there is no proposal for a new syntax to introduce a type name.

Function templates

The approach is simple: allow `auto` parameters to produce function templates (as they produce polymorphic lambdas), and allow the `auto` to be preceded by a concept name. In every case, such a parameter is a deduced parameter, and we can see which parameters are deduced and which ones are not:

```
[](auto a, auto& b, const auto& c, auto&& d) {...}; // unconstrained
[](Constraint auto a, Constraint auto& b, const Constraint auto& c, Constraint auto&& d) {...}; // constrain
```

```
void f1(auto a, auto& b, const auto& c, auto&& d) {...}; // unconstrained
void f2(Constraint auto a, Constraint auto& b, const Constraint auto& c, Constraint auto&& d) {...}; // cons

[](Constraint auto&& a, SomethingElse&& b) {...}; // a constrained deduced forwarding reference and a concre
void f3(Constraint auto&& a, SomethingElse&& b) {...}; // a constrained deduced forwarding reference and a c
```

The appearance of `auto` (including `Constraint auto`) in a parameter list tells us that we are dealing with a function template. For each parameter, we know whether it is deduced or not. We can tell apart concepts from types: concepts precede `auto`, types do not.

Return types and variable declarations

Constrained return types work the same way:

```
auto f4();                  // unconstrained, deduced.

Constraint auto f5();       // constrained, deduced.

Whatever f6();              // See part 2. If Whatever is a type, not deduced.
                            // If Whatever is a concept, constrained and deduced.
```

Note that `f4`, `f5` and `f6` are not templates (whereas the previous `f1`, `f2` and `f3` *are* templates). Here, there is no mention of `auto` in the parameter list. Users have the choice of adopting a style where it is explicit as to whether the return type is deduced.

Constrained types for variables work the same way:

```
auto x1 = f1();             // unconstrained, deduced.

Constraint auto x2 = f2();  // constrained, deduced.

Whatever x3 = f3();         // See part 2. If Whatever is a type, not deduced.
                            // If Whatever is a concept, constrained and deduced.
```

Again, users can make it so that it is easy to see when deduction occurs.

Since non-type template parameters can be deduced via `auto` (as in `template <auto N> void f();`), we also allow a constraint there:

```
template <Constraint auto N> void f7();
```

Note, however, that this can only be a type constraint; non-type concepts (including auto concepts) are not allowed in this form.

Other uses of `auto`

In concert with the general approach that "`Constraint auto` goes wherever `auto` goes", new-expressions and conversion operators work:

```
auto alloc_next() { return new Sortable auto(this->next_val()); }

operator Sortable auto() { }
```

A "`Constraint auto`" cannot be used to indicate that a function declarator has a trailing return type:

```
Constraint auto f() -> auto; // ill-formed; shall be the single type-specifier auto
```

`decltype(auto)` can also be constrained:

```
auto f() -> Constraint decltype(auto);
Constraint decltype(auto) x = f();
```

Structured bindings do deduce `auto` in some cases; however, the `auto` is deduced from the whole (and not from the individual components). It is somewhat doubtful that applying the constraint to the whole, as opposed to (for example) applying separately to each component, is the correct semantic. Therefore, we propose to defer enabling the application of constraints to structured bindings to separate papers.

General rules

The constraint applies directly to the deduced type. It does not apply to the possibly cv-qualified type described by the type specifiers, nor does it apply to the type declared for the variable:

```
const Assignable<int> auto&& c = *static_cast<int *>(p); // Assignable<int &, int>
```

Naturally, if the deduced type is cv-qualified (or a reference), the constraint applies to that type.

To keep things simple, an `auto` (or `decltype(auto)`) being constrained is always immediately preceded by the constraint. So, cv-qualifiers and concept-identifiers cannot be freely mixed:

```
const Contraint auto x = foo(); // ok
Constraint const auto x = foo(); // ill-formed
Constraint auto const y = foo(); // ok
```

We propose only the ability to apply one single constraint for a parameter, return type, or non-type template parameter. Any proposal to consider multiple constraints should happen separately after C++20.

Partial concept identifiers also work. Given a concept `template <typename T, typename... Args> concept Constructible = /* ... */;`, we can say:

```
void f(Constructible<int> auto x);   // Constructible<decltype(x), int> is satisfied

Constructible<int> auto f();

Constructible<int> auto x = f();

template <Constructible<int> auto N> void f();
```

~~Part 2: Relaxed "constrained auto"~~ [not proposed]

Part 3: Meaning of "`template <Concept T>`"

In [temp.param]/10 we have:

> A *constrained-parameter* declares a template parameter whose kind (type, non-type, template) and type match that of the prototype parameter (17.6.8) of the concept designated by the *type-constraint* in the *constrained-parameter*. Let `X` be the prototype parameter of the designated concept. The declared template parameter is determined by the kind of `X` (type, non-type, template) and the optional ellipsis in the *constrained-parameter* as follows.
>
> - If `X` is a type *template-parameter*, the declared parameter is a type *template-parameter*.
> - If `X` is a non-type *template-parameter*, the declared parameter is a non-type *template-parameter* having the same type as `X`.
> - If `X` is a template *template-parameter*, the declared parameter is a template *template-parameter* having the same *template-parameter-list* as `X`, excluding default template arguments.
> - If the *type-constraint* is followed by an ellipsis, then the declared parameter is a template parameter pack (17.6.3).
>
> [*Example*:
>
> ```
> template<typename T> concept C1 = true;
> template<template<typename> class X> concept C2 = true;
> template<int N> concept C3 = true;
> template<typename... Ts> concept C4 = true;
> template<char... Cs> concept C5 = true;
>
> template<C1 T> void f1();       // OK, T is a type template-parameter
> template<C2 X> void f2();       // OK, X is a template with one type-parameter
> template<C3 N> void f3();       // OK, N has type int
> template<C4... Ts> void f4();   // OK, Ts is a template parameter pack of types
> template<C4 T> void f5();       // OK, T is a type template-parameter
> template<C5... Cs> void f6();   // OK, Cs is a template parameter pack of chars
> ```
>
> —*end example*]

Does that seem like a mouthful?

That's because it is. In `template <Constraint T>`, the kind of `T` depends on the kind of the prototype parameter of `Constraint`.

We instead propose that, for such a constrained-parameter syntax:

- `T` should always be a type, and
- `Constraint` would always need to be a concept that has a corresponding type parameter or type parameter pack.

To be clear, we are not proposing that concepts in general should not have non-type or template template parameters. We are merely proposing for it to be the case that the constrained parameter shortcut is not provided for concepts with such prototype parameters; such concepts would need to be used with a *requires-clause*. The constrained parameter syntax should mean just one thing. Note that the same syntax `template <A T>` is still a non-type parameter when `A` is a type name rather than a concept. We are willing to tolerate this small potential for ambiguity.

The rationale for this part is as follows:

1. It seems desirable to have the constrained template parameter syntax.
2. It would be nice if that syntax covered the most common case.
3. It would further be nice if that syntax covered *only* the most common case.
4. The other cases are expected to be so rare that there's no need to provide a shortcut for them, and they are certainly rare enough that they shouldn't use the same syntax.

So, to clarify:

- `template <MyIntTypeDef N>` means a non-type parameter, like it always did.
- `template <ConceptName T>` means a type parameter constrained by `ConceptName`, and the prototype parameter of `ConceptName` needs to be a type parameter or a type parameter pack.
- `template <auto N>` means a non-type parameter with a deduced type.
- `template <ConceptName auto N>` means a non-type parameter with a deduced type constrained by `ConceptName`, and the prototype parameter of `ConceptName` needs to be a type parameter or a type parameter pack.

Other use cases can be done with *requires-clause*s.

Part 4: Meaning of "`template <Concept... T>`" and its friends

In [temp.param]/11 we have:

> `template<C2... T> struct s3; // associates C2<T...>`

This seems to be doing an unexpected thing, which is having the constraint apply to more than one type in a pack at a time. We propose that, regardless of whether the prototype parameter of the named concept is a pack:

- For a simple pack of constrained types, the concept mentioned is applied, as a unary concept, to each type in the pack in turn.
- For a pack of constrained types that use *partial-concept-id*s, the concept mentioned is applied, as an n-ary concept whose arity is unaffected by the size of the pack, *individually* to each type in the pack in turn.

In other words,

- `template <ConceptName... T> void f(T...);` means a variadic function template where each type in the pack `T` needs to satisfy `ConceptName` as a unary concept, applied as `ConceptName<`T_n`>`.
- Similarly, `void f(ConceptName auto... T);` means exactly the same thing.
- `template <ConceptName<int>... U> void f(U...);` means a variadic function template where each type in the pack `U` needs to satisfy `ConceptName` as a binary concept, applied as `ConceptName<`U_n`, int>`.
- Similarly, `void f(ConceptName<int> auto... U);` means exactly the same thing.
- `template <ConceptName<0u, void, wchar_t>... U> void f(U...);` means a variadic function template where each type in the pack `U` needs to satisfy `ConceptName` as a n-ary concept, applied as `ConceptName<`U_n`, 0u, void, wchar_t>`.

~~Part 5: Meaning of "`-> Concept auto`" and its friends~~ [not proposed]

Proposed wording for Parts 1, 3, and 4

Changes in [expr]

Update [expr.prim.lambda, 7.5.5], paragraph 5, to allow placeholder type specifiers as lambda parameters.

> A lambda is a generic lambda if ~~the auto *type-specifier* appears as one of the *decl-specifiers*~~there is a *decl-specifier* that is a *placeholder-type-specifier* in the *decl-specifier-seq* of a *parameter-declaration* of the *lambda-expression*, or if the lambda has a *template-parameter-list*. [*Example:* [...] *—end example*]

In [expr.prim.lambda.closure, 7.5.5.1], modify paragraph 3.

> The closure type for a ~~non-generic~~ *lambda-expression* has a public inline function call operator (for a non-generic lambda) or function call operator template (for a generic lambda) (11.5.4) whose parameters and return type are described by the *lambda-expression*'s *parameter-declaration-clause* and *trailing-return-type* respectively~~. For a generic lambda, the closure type has a public inline function call operator member template (12.6.2)~~ , and whose *template-parameter-list* consists of the specified *template-parameter-list*, if any~~, to which is appended one invented type *template-parameter* for each occurrence of `auto` in the lambda's *parameter-declaration-clause*, in order of appearance. The invented type *template-parameter* is a template parameter pack if the corresponding parameter-declaration declares a function parameter pack (9.2.3.5). The return type and function parameters of the function call operator template are derived from the *lambda-expression*'s *trailing-return-type* and *parameter-declaration-clause* by replacing each occurrence of `auto` in the *decl-specifier*s of the *parameter-declaration-clause* with the name of the corresponding *invented template-parameter*~~. The *requires-clause* of the function call operator template is the *requires-clause* immediately following < *template-parameter-list* >, if any. The trailing *requires-clause* of the function call operator or operator template is the *requires-clause* following the *lambda-declarator*, if any. [*Note*: The function call operator for a generic lambda might be an abbreviated function template (9.2.3.5). *—end note*] [*Example:* [...] *—end example*]

Modify paragraph 6 as follows.

> [*Note*: The function call operator or operator template may be constrained (12.4.2) by a ~~*constrained-parameter*~~*type-constraint* (12.1), a *requires-clause* (Clause 12), or a trailing *requires-clause* (9.2). [*Example*:
>
> ```
> template <typename T> concept C1 = /* ... */;
> template <std::size_t N> concept C2 = /* ... */;
> template <typename A, typename B> concept C3 = /* ... */;
>
> auto f = []<typename T1, C1 T2> requires C2<sizeof(T1) + sizeof(T2)>
>          (T1 a1, T1 b1, T2 a2, auto a3, auto a4) requires C3<decltype(a4), T2> {
>   // T2 is ~~a constrained parameter~~constrained by a type-constraint,
>   // T1 and T2 are constrained by a requires-clause, and
>   // T2 and the type of a4 are constrained by a trailing requires-clause.
> };
> ```
>
> *—end example*] *—end note*]

Changes in [dcl]

Change [dcl.type.simple, 9.1.7.2] paragraph 1 to add *placeholder-type-specifier*s.

> *simple-type-specifier*:
>
> *nested-name-specifier*$_{opt}$ *type-name*
>
> *nested-name-specifier* `template` *simple-template-id*
>
> *nested-name-specifier*$_{opt}$ *template-name*
>
> `char`
>
> `char16_t`
>
> `char32_t`
>
> `wchar_t`
>
> `bool`
>
> `short`
>
> `int`
>
> `long`
>
> `signed`
>
> `unsigned`

> float
> double
> void
> ~~auto~~
> *decltype-specifier*
> *placeholder-type-specifier*
>
> *type-name*:
> class-name
> enum-name
> typedef-name
> simple-template-id
>
> *decltype-specifier*:
> decltype (expression)
> ~~decltype (auto)~~
>
> *placeholder-type-specifier:*
> *type-constraint*$_{opt}$ auto
> *type-constraint*$_{opt}$ decltype (auto)

Modify paragraph 2 as follows.

> ~~The *simple-type-specifier* auto~~A *placeholder-type-specifier* is a placeholder for a type to be deduced (9.1.7.4).

Add *placeholder-type-specifier*s to the table of *simple-type-specifier*s and their meaning.

> | Specifier(s) | Type |
> |---|---|
> | *type-name* | the type named |
> | *simple-template-id* | the as defined in 12.2 |
> | ... | ... |
> | void | "void" |
> | ~~auto~~ | ~~placeholder for a type to be deduced~~ |
> | ~~decltype(auto)~~ | ~~placeholder for a type to be deduced~~ |
> | decltype(*expression*) | the type as described below |
> | *placeholder-type-specifier* | placeholder for a type to be deduced |

In [dcl.spec.auto, 9.1.7.4], modify and split paragraph 1 as follows.

> ~~The auto and decltype(auto) *type-specifier*s are used to~~A *placeholder-type-specifier* designates a placeholder type that will be replaced later by deduction from an initializer.
>
> A *placeholder-type-specifier* of the form *type-constraint*$_{opt}$ auto can be used in the *decl-specifier-seq* of a *parameter-declaration* of a function declaration or *lambda-expression* and signifies that the function is an abbreviated function template (9.2.3.5) or the~~The auto *type-specifier* is also used to introduce a function type having a *trailing-return-type* or to signify that a~~ lambda is a generic lambda (7.5.5). ~~The auto *type-specifier* is also used to introduce a structured binding declaration (9.5).~~

Modify (old) paragraph 3 as follows.

> The type of a variable declared using ~~auto or decltype(auto)~~a placeholder type is deduced from its initializer. This use is allowed in an initializing declaration (9.3) of a variable. ~~auto or decltype(auto)~~The placeholder type shall appear as one of the *decl-specifier*s in the *decl-specifier-seq* and the *decl-specifier-seq* shall be followed by one or more declarators, each of which shall be followed by a non-empty initializer. [...]—*end example*] The auto *type-specifier* can also be used to introduce a structured binding declaration (9.5).

Modify (old) paragraph 5 as follows.

> A program that uses ~~auto or decltype(auto)~~a placeholder type in a context not explicitly allowed in this subclause is ill-formed.

In [dcl.type.auto.deduct, 9.1.7.4.1], modify the last sentence of paragraph 2 as follows.

> [...] In the case of a return statement with no operand or with an operand of type `void`, `T` shall be either *type-constraint*$_{opt}$ `decltype(auto)` or *cv* *type-constraint*$_{opt}$ `auto`.

Modify paragraph 4 as follows.

> If the ~~placeholder is the `auto` *type-specifier*~~*placeholder-type-specifier* is of the form *type-constraint*$_{opt}$ `auto`, the deduced type `T'` replacing `T` is determined using the rules for template argument deduction. Obtain `P` from `T` by replacing the occurrences of *type-constraint*$_{opt}$ `auto` with either a new invented type template parameter `U` or, if the initialization is copy-list-initialization, with `std::initializer_list<U>`. [...]

Modify paragraph 5 as follows.

> If the ~~placeholder is the `decltype(auto)` *type-specifier*~~*placeholder-type-specifier* is of the form *type-constraint*$_{opt}$ `decltype(auto)`, `T` shall be the placeholder alone. The type deduced for `T` is determined [...]

Append a new paragraph as follows.

> ?. For a *placeholder-type-specifier* with a *type-constraint*, if the type deduced for the placeholder does not satisfy its immediately-declared constraint ([temp, 12]), the program is ill-formed.

Add the following paragraphs to [dcl.fct, 9.2.3.5], after paragraph 16.

> ?. An *abbreviated function template* is a function declaration whose parameter-type-list includes one or more placeholders (9.1.7.4). An abbreviated function template is equivalent to a function template (17.6.5) whose *template-parameter-list* includes one invented type *template-parameter* for each occurrence of a placeholder type in the *decl-specifier-seq* of a *parameter-declaration* in the function's parameter-type-list, in order of appearance. For a *placeholder-type-specifier* of the form `auto`, the invented parameter is an unconstrained *type-parameter*. For a *placeholder-type-specifier* of the form *type-constraint* `auto`, the invented parameter is a *type-parameter* with that *type-constraint*. The invented type *template-parameter* is a template parameter pack if the corresponding parameter-declaration declares a function parameter pack (9.2.3.5). If the placeholder contains `decltype(auto)`, the program is ill-formed. The adjusted function parameters of an abbreviated function template are derived from the *parameter-declaration-clause* by replacing each occurrence of a placeholder with the name of the corresponding invented *template-parameter*.
>
> [*Example*:
>
> ```
> template<typename T>      concept C1 = /* ... */;
> template<typename T>      concept C2 = /* ... */;
> template<typename... Ts> concept C4 = /* ... */;
>
> void g1(const C1 auto*, C2 auto&);
> void g3(C1 auto&...);
> void g5(C4 auto...);
> void g7(C4 auto);
> ```
>
> These declarations are functionally equivalent (but not equivalent) to the following declarations.
>
> ```
> template<C1 T, C2 U> void g1(const T*, U&);
> template<C1... Ts>   void g3(Ts&...);
> template<C4... Ts>   void g5(Ts...);
> template<C4 T>       void g7(T);
> ```
>
> Abbreviated function templates can be specialized like all function templates.
>
> ```
> template<> void g1<int>(const int*, const double&);   // OK, specialization of g1<int, const double>
> ```
>
> *—end example*]
>
> ?. An abbreviated function template can have a *template-head*. The invented *template-parameter*s are appended to the *template-parameter-list* after the explicitly declared *template-parameter*s.
>
> [*Example*:

```
template<typename> concept C = /* ... */;

template <typename T, C U>
  void g(T x, U y, C auto z);
```

This is functionally equivalent to each of the following two declarations.

```
template<typename T, C U, C W>
  void g(T x, U y, W z);

template<typename T, typename U, typename W>
  requires C<U> && C<W>
  void g(T x, U y, W z);
```

—end example]

?. A function declaration at block scope shall not declare an abbreviated function template.

Changes in [temp]

Add to the grammar in [temp, 12] the following.

concept-name:
identifier

type-constraint:
nested-name-specifier$_{opt}$ *concept-name*
nested-name-specifier$_{opt}$ *concept-name* < *template-argument-list*$_{opt}$ >

Append a new paragraph as follows.

?. A *type-constraint* `Q` that designates a concept `C` can be used to constrain a contextually-determined type or template type parameter pack `T` with a *constraint-expression* `E` defined as follows. If `Q` is of the form `C<A1, ..., An>`, then let `E′` be `C<T, A1, ..., An>`. Otherwise, let `E′` be `C<T>`. If `T` is not a pack, then `E` is `E′`, otherwise `E` is `(E′ && ...)`. This is called the *immediately-declared constraint* of `T`. The concept designated by a *type-constraint* shall be a type concept (12.6.8).

Change [temp.param, 12.1] paragraph 1 to remove the grammar for *constrained-parameter* and to enhance the grammar of *type-parameter*.

Editorial note: No further appearances of "*qualified-concept-name*" should remain in the working draft after application of P1084R2 and P1141R2 (this paper).

template-parameter:
type-parameter
parameter-declaration
~~*constrained-parameter*~~

type-parameter:
type-parameter-key ...$_{opt}$ *identifier*$_{opt}$
type-parameter-key *identifier*$_{opt}$ = *type-id*
type-constraint ...$_{opt}$ *identifier*$_{opt}$
type-constraint *identifier*$_{opt}$ = *type-id*
template-head *type-parameter-key* ...$_{opt}$ *identifier*$_{opt}$
template-head *type-parameter-key* *identifier*$_{opt}$ = *id-expression*

type-parameter-key:
class
typename

~~*constrained-parameter:*~~
~~*qualified-concept-name* ... *identifier*$_{opt}$~~

~~*qualified-concept-name identifier*$_{opt}$ *default-template-argument*$_{opt}$~~

~~*qualified-concept-name:*~~
~~*nested-name-specifier*$_{opt}$ *concept-name nested-name-specifier*$_{opt}$*partial-concept-id*~~

~~*partial-concept-id:*~~
~~*concept-name* < *template-argument-list*$_{opt}$ >~~

Change [12.1, temp.param] paragraph 9 as follows.

~~A *partial-concept-id* is a *concept-name* followed by a sequence of *template-arguments*. These template arguments are used to form a *constraint-expression* as described below.~~A *type-parameter* that starts with a *type-constraint* introduces the immediately-declared constraint of the parameter.

Delete [12.1, temp.param] paragraph 10.

~~A *constrained-parameter* declares a template parameter whose kind (type, non-type, template) and type match that of the prototype parameter (12.6.8) of the concept designated by the *qualified-concept-name* in the *constrained-parameter*. Let X be the prototype parameter of the designated concept. The declared template parameter is determined by the kind of X (type, non-type, template) and the optional ellipsis in the *constrained-parameter* as follows.~~

- ~~If X is a type *template-parameter*, the declared parameter is a type *template-parameter*.~~
- ~~If X is a non-type *template-parameter*, the declared parameter is a non-type *template-parameter* having the same type as X.~~
- ~~If X is a template *template-parameter*, the declared parameter is a template *template-parameter* having the same *template-parameter-list* as X, excluding default template arguments.~~
- ~~If the *qualified-concept-name* is followed by an ellipsis, then the declared parameter is a template parameter pack (temp.variadic, 12.6.3).~~

~~[*Example*:~~

```
template<typename T> concept C1 = true;
template<template<typename> class X> concept C2 = true;
template<int N> concept C3 = true;
template<typename... Ts> concept C4 = true;
template<char... Cs> concept C5 = true;

template<C1 T> void f1();           // OK, T is a type template-parameter
template<C2 X> void f2();           // OK, X is a template with one type-parameter
template<C3 N> void f3();           // OK, N has type int
template<C4... Ts> void f4();       // OK, Ts is a template parameter pack of types
template<C4 T> void f5();           // OK, T is a type template-parameter
template<C5... Cs> void f6();       // OK, Cs is a template parameter pack of chars
```

~~—*end example*]~~

In [12.1, temp.param], delete the normative wording of (old) paragraph 11 and merge the (modified) example into paragraph 9 as follows.

Editorial note: This change effects the design change of Part 4 (changing the meaning of `...`). The new pack expansion behaviour is subsumed by the “immediately-declared constraint” facility.

~~A *constrained-parameter constraint-expression*. The expression is derived from the *qualified-concept-name* Q in the *constrained-parameter*, its designated concept C, and the declared template parameter P.~~

- ~~First, a template argument A is invented from P. If P declares a template parameter pack ([temp.variadic]) and C is a variadic concept ([temp.concept]), then A is the pack expansion P.... Otherwise, A is the *id-expression* P.~~
- ~~Then, an *id-expression* E is formed as follows. If Q is a *concept-name*, then E is C<A>. Otherwise, Q is a *partial-concept-id* of the form C<A1, A2, ..., An>, and E is C<A, A1, A2, ..., An>.~~
- ~~Finally, if P declares a template parameter pack and C is not a variadic concept, E is adjusted to be the *fold-expression* (E && ...) (7.5.6).~~

~~E is the introduced *constraint-expression*.~~

[*Example*:

```
template<typename T> concept C1 = true;
template<typename... Ts> concept C2 = true;
template<typename T, typename U> concept C3 = true;

template<C1 T> struct s1;          // associates C1<T>
template<C1... T> struct s2;       // associates (C1<T> && ...)
template<C2... T> struct s3;       // associates ~~C2<T...>~~(C2<T> && ...)
template<C3<int> T> struct s4;     // associates C3<T, int>
template<C3<int>... T> struct s5;  // associates (C3<T, int> && ...)
```

—*end example*]

Insert a new paragraph after (old) paragraph 11.

?. A non-type template parameter declared with a type that contains a placeholder type with a *type-constraint* introduces the immediately-declared constraint of the invented type corresponding to the placeholder.

Delete (old) paragraph 13.

~~The default *template-argument* of a *constrained-parameter* shall match the kind (type, non-type, template) of the declared template parameter. [*Example*. […] —*end example*]~~

Modify (old) paragraph 19.

If a *template-parameter* is a *type-parameter* with an ellipsis prior to its optional identifier or is a *parameter-declaration* that declares a pack (9.2.3.5), then the *template-parameter* is a template parameter pack (12.6.3). A template parameter pack that is a *parameter-declaration* whose type contains one or more unexpanded packs is a pack expansion. Similarly, a template parameter pack that is a *type-parameter* with a *template-parameter-list* containing one or more unexpanded packs is a pack expansion. A type parameter pack with a *type-constraint* that contains an unexpanded parameter pack is a pack expansion. A template parameter pack that is a pack expansion shall not expand a template parameter pack declared in the same template-parameter-list.

Modify [temp.constr.decl, 12.4.2] paragraph 2 as follows.

Constraints can also be associated with a declaration through the use of ~~*constrained-parameters*~~*type-constraints* in a *template-parameter-list*. Each of these forms introduces additional *constraint-expression*s that are used to constrain the declaration.

Modify paragraph 3 as follows.

A template's *associated constraints* are defined as follows:

- […]
- Otherwise, the associated constraints are the normal form of a logical AND expression (7.6.14) whose operands are in the following order:
  - the *constraint-expression* introduced by each ~~*constrained-parameter*~~*type-constraint* (12.1) in the declaration's template-parameter-list, in order of appearance, and
  - the *constraint-expression* introduced by a *requires-clause* following a template-parameter-list (Clause12), and
  - the *constraint-expression* introduced by *type-constraint* in the parameter-type-list of a function declaration
  - the *constraint-expression* introduced by a *trailing requires-clause* (9.2) of a function declaration (9.2.3.5).

Modify [temp.decls, 12.6] paragraph 2 as follows.

For purposes of name lookup and instantiation, default arguments, ~~*partial-concept-ids*~~*type-constraints*, *requires-clause*s (Clause 12), and *noexcept-specifier*s of function templates and of member functions of class templates are considered definitions; each default argument, ~~*partial-concept-ids*~~*type-constraint*, *requires-clause*, or *noexcept-specifier* is a separate definition which is unrelated to the templated function

> definition or to any other default arguments~~ *partial-concept-ids*~~, *type-constraints*, *requires-clause*s, or *noexcept-specifier*s. For the purpose of instantiation, the substatements of a constexpr if statement (8.4.1) are considered definitions.

Modify [temp.variadic, 12.6.3] paragraph 5 bullet (5.3.2) as follows.

> A *pack expansion* consists of [...]
>
> - [...]
> - In a template parameter pack that is a pack expansion (12.1):
>   - if the template parameter pack is a *parameter-declaration*; the pattern is the parameter-declaration without the ellipsis;
>   - if the template parameter pack is a *type-parameter*~~ with a *template-parameter-list*~~; the pattern is the corresponding *type-parameter* without the ellipsis.
> - In an *initializer-list* (9.3); the pattern is an *initializer-clause*.
> - [...]

Modify the example in [temp.concept, 12.6.8] paragraph 2 as follows.

> ```
> ...
> template<C T>  // C, as a type-constraint, constrains f2(T)~~ as a constrained-parameter~~
> ...
> ```

Modify paragraph 6 as follows.

> The first declared template parameter of a concept definition is its *prototype parameter*. A *type concept* is a concept whose prototype parameter is a type *template-parameter*.~~A *variadic concept* is a concept whose prototype parameter is a template parameter pack.~~

Modify [temp.res, 12.7] paragraph 8 item (8.2) as follows.

> - no valid specialization can be generated for a template or a substatement of a constexpr if statement (8.4.1) within a template and the template is not instantiated, or
> - no substitution of template arguments into a ~~*partial-concept-id*~~*type-constraint* or *requires-clause* would result in a valid expression, or
> - every valid specialization of a variadic template requires an empty template parameter pack, or
> - [...]

Modify the note in [temp.inst, 12.8.1] paragraph 1 as follows.

> [...] [*Note*: Within a template declaration, a local class (10.5) or enumeration and the members of a local class are never considered to be entities that can be separately instantiated (this includes their default arguments, *noexcept-specifier*s, and non-static data member initializers, if any, but not their ~~*partial-concept-ids*~~*type-constraints* or *requires-clause*s). [...]

Modify paragarph 17 as follows.

> The ~~*partial-concept-ids*~~*type-constraints* and *requires-clause* of a template specialization or member function are not instantiated along with the specialization or function itself, even for a member function of a local class; substitution into the atomic constraints [...]